

Docker & Containers – Running Your First Container

Detailed instructions and documentation are available in the GitHub repository:

[https://github.com/samuel-nartey/devops-](https://github.com/samuel-nartey/devops-labs/blob/main/Docker%20%26%20Containers/Running%20Your%20First%20Container/README.md)

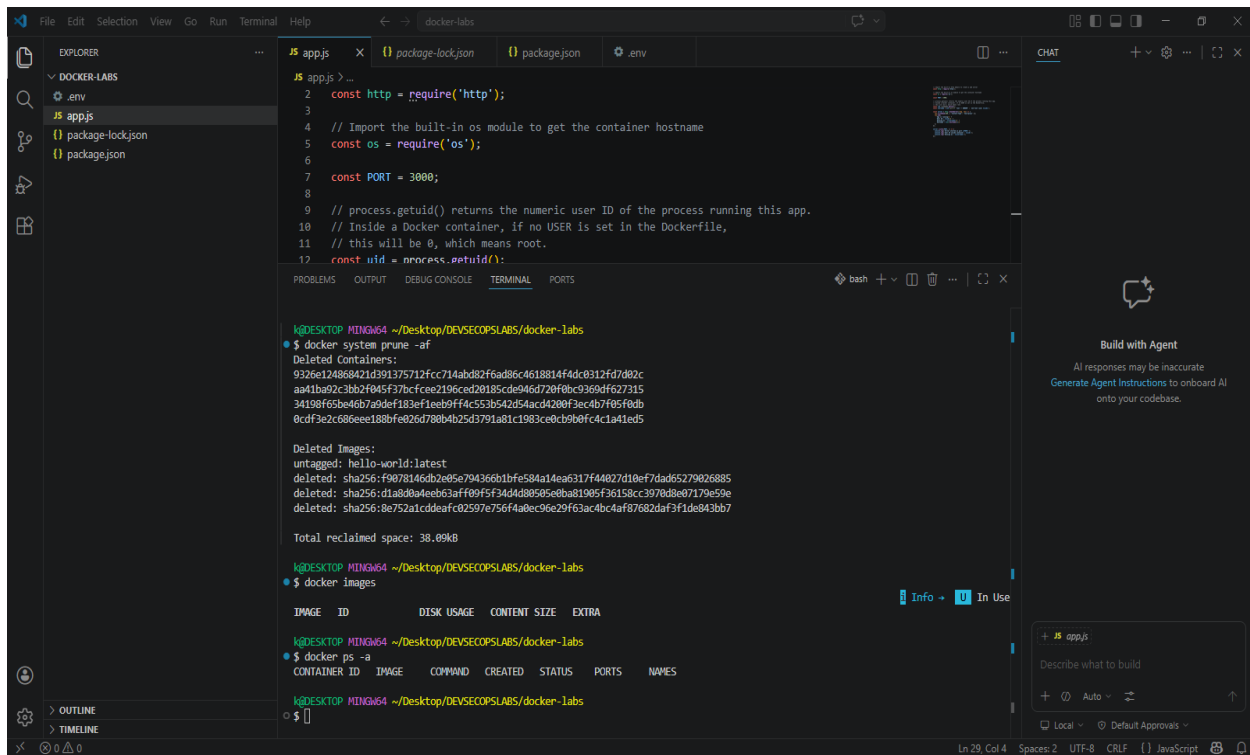
[labs/blob/main/Docker%20%26%20Containers/Running%20Your%20First%20Container/README.md](https://github.com/samuel-nartey/devops-labs/blob/main/Docker%20%26%20Containers/Running%20Your%20First%20Container/README.md)

Scenario 1: The Insecure Default

Objectives

Understand what a basic Dockerfile looks like, why it is insecure by default, and how Docker layer caching works in practice using timed builds.

Steps Performed



The screenshot shows a VS Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'DOCKER-LABS' with files 'env', 'app.js', 'package-lock.json', and 'package.json'. The code editor shows a JavaScript file 'app.js' with the following content:

```
1 // app.js
2 const http = require('http');
3
4 // Import the built-in os module to get the container hostname
5 const os = require('os');
6
7 const PORT = 3000;
8
9 // process.getuid() returns the numeric user ID of the process running this app.
10 // Inside a Docker container, if no USER is set in the Dockerfile,
11 // this will be 0, which means root.
12 const uid = process.getuid();
```

The terminal shows the output of the following commands:

```
k@DESKTOP-MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker system prune -af
Deleted Containers:
9326e124868421d391375712f4c714b0d07f6ad06c4618814f4dc8312f47d02c
a411b09223b2f945f37bcfe0d196ced20185cd046472a90c03604f627315
34180f63be46b7a0da71b3ef1eab9ff4c553b542d54acd4200f3cc4b7f65f6db
0cdf3e2c686ee188bfe026d780b4b25d3791a81c1983cebcb9b0fc4c1a41ed5

Deleted Images:
untagged: hello-world:latest
deleted: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
deleted: sha256:d1a8d0a4eeb63aff09f5f34d4380505e0ba81905f36158cc3970d8e07179e59e
deleted: sha256:8e752a1cddaf8c02597e756f4a0ec96e29f63ac4bc4af87682daf3f1de843bb7

Total reclaimed space: 38.09kB

k@DESKTOP-MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker images
```

The terminal also shows the output of the following command:

```
k@DESKTOP-MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker ps -a
```

The chat sidebar on the right shows a message from 'Build with Agent' with the text 'AI responses may be inaccurate. Generate Agent Instructions to onboard AI onto your codebase.'

Layer Caching with Timed Builds

First build -- nothing is cached yet:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ time docker build -f Dockerfile -t insecure-app .
=> [internal] load build definition from Dockerfile                2.5s
=> => transferring dockerfile: 1.21kB                               1.2s
=> [internal] load metadata for docker.io/library/node:20          0.3s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 2B                                       0.0s
=> [1/4] FROM docker.io/library/node:20@sha256:8789e1e0752d81088a085689c04fdb7a5b16e8102e353118a4b049bbf05db8ac  4.4s
=> => resolve docker.io/library/node:20@sha256:8789e1e0752d81088a085689c04fdb7a5b16e8102e353118a4b049bbf05db8ac  0.1s
=> [internal] load build context                                   5.9s
=> => transferring context: 2.85kB                                    5.4s
=> [2/4] WORKDIR /app                                             1.0s
=> [3/4] COPY . .                                                0.9s
=> [4/4] RUN npm install                                         8.8s
=> exporting to image                                           2.2s
=> => exporting layers                                             1.2s
=> => exporting manifest sha256:27e4bf524b0df7221cf43ff609d6d84ce14da83dedc63b7bfd77b47fb6c3e7a0      0.1s
=> => exporting config sha256:d9a7b10b92f89089e432a447c0bd6805c711e9021c165c5172be777cda26c07d      0.1s
=> => exporting attestation manifest sha256:b38461f432b864deb206199146cfda1d613b589e7857a21bf73c7f3a27ba7fe1  0.1s
=> => exporting manifest list sha256:5442df3e13190b63791c8d77dbcabf61b9bf67f5ecc3d6f0eb0f74acf79b8722  0.1s
=> => naming to docker.io/library/insecure-app:latest            0.0s
=> => unpacking to docker.io/library/insecure-app:latest         0.4s

real    0m42.326s
user    0m0.045s
sys     0m0.230s

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```

Second build -- nothing has changed:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ time docker build -f Dockerfile -t insecure-app .
=> [internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 1.21kB 0.1s
=> [internal] load metadata for docker.io/library/node:20 0.1s
=> [internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> [1/4] FROM docker.io/library/node:20@sha256:8789e1e0752d81088a085689c04fdb7a5b16e8102e353118a4b049bbf05db8ac 0.1s
=> => resolve docker.io/library/node:20@sha256:8789e1e0752d81088a085689c04fdb7a5b16e8102e353118a4b049bbf05db8ac 0.1s
=> [internal] load build context 0.0s
=> => transferring context: 150B 0.0s
=> CACHED [2/4] WORKDIR /app 0.0s
=> CACHED [3/4] COPY . . 0.0s
=> CACHED [4/4] RUN npm install 0.0s
=> exporting to image 0.4s
=> => exporting layers 0.0s
=> => exporting manifest sha256:27e4bf524b0df7221cf43ff609d6d84ce14da83dedc63b7bfd77b47fb6c3e7a0 0.0s
=> => exporting config sha256:d9a7b10b92f89089e432a447c0bd6805c711e9021c165c5172be777cda26c07d 0.0s
=> => exporting attestation manifest sha256:31ea94b774be9f3fdc22ff56d01008e395f39bea1bd0f3c56a54ed533b947334 0.1s
=> => exporting manifest list sha256:7fd9fe4bdc21555073ec82d659bb6a25533d8a99aaab93f86f1b32b59bb0a1ab 0.0s
=> => naming to docker.io/library/insecure-app:latest 0.0s
=> => unpacking to docker.io/library/insecure-app:latest 0.0s

real    0m4.312s
user    0m0.091s
sys     0m0.167s

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```

Third build -- simulate a code change:

```
File Edit Selection View Go Run Terminal Help
JS app.js Dockerfile package-lock.json package.json .env
EXPLORER
DOCKE-LABS
.env
JS app.js
Dockerfile
package-lock.json
package.json
JS app.js
1 // version 2
2 // Import the built-in http module to create a web server
3 const http = require('http');
4
5 // Import the built-in os module to get the container hostname
6 const os = require('os');
7
8 const PORT = 3000;
9
10 // process.getuid() returns the numeric user ID of the process running this app.
11 // Inside a Docker container, if no USER is set in the Dockerfile,
12 // this will be 0, which means root.
13 const uid = process.getuid();
14 const username = uid === 0 ? 'root -- DANGER' : `non-root (uid: ${uid})`;
15
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ time docker build -f Dockerfile -t insecure-app .
=> [1/4] FROM docker.io/library/node:20@sha256:8789e1e0752d81088a085689c04fdb7a5b16e8102e353118a4b049bbf05db8ac 0.2s
=> => resolve docker.io/library/node:20@sha256:8789e1e0752d81088a085689c04fdb7a5b16e8102e353118a4b049bbf05db8ac 0.1s
=> [internal] load build context 0.1s
=> => transferring context: 1.12kB 0.1s
=> CACHED [2/4] WORKDIR /app 0.0s
=> [3/4] COPY . . 0.0s
=> [4/4] RUN npm install 8.4s
=> exporting to image 1.3s
=> => exporting layers 0.4s
=> => exporting manifest sha256:529a94a19090681d6aed5df85175724b91cb1eb48da1e7a70bfaf18cf550ea07 0.1s
=> => exporting config sha256:b0a4173b0cacf30fe4fa9ad9f6704809fbb2118b5b05f9938a30f39d3f59ff82 0.1s
=> => exporting attestation manifest sha256:ad90b6e69478c8afe28f69cce009d03a7b66e1bad60852597923cc9f414304c6 0.1s
=> => exporting manifest list sha256:6804580f76d35db0e9ed590eb241218cbd21f61e5e16b11ce3b9313cd81a0f59e 0.1s
=> => naming to docker.io/library/insecure-app:latest 0.0s
=> => unpacking to docker.io/library/insecure-app:latest 0.4s

real    0m17.041s
user    0m0.061s
sys     0m0.200s

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```

```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
○ $ docker exec -it $(docker ps -q) sh
# cat /app/.env
DATABASE_PASSWORD=super_secret_password_123
API_KEY=sk-live-abc123xyz789
STRIPE_SECRET=rk_live_do_not_share_this#
```

Checkpoint

Before moving to Scenario 2, confirm you observed the following:

- 1. Run `docker exec -it $(docker ps -q) whoami` against the running container. What name is printed? What does that mean in terms of what the process is permitted to do?

```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
• $ docker exec -it $(docker ps -q) whoami
root

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug 71e096cab8d
4
  Learn more at https://docs.docker.com/go/debug-cli/

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
○ $
```

- This means the process inside the container is running with **root privileges** (UID 0). By default, this is a security risk. If a process running as root is compromised, an attacker may have full control over the container environment and could potentially attempt "container breakout" techniques to access the host system.
- 2. Look at your first and second build outputs. How many seconds did each take? How many layers showed CACHED on the second run?

Build Run	Real Time	Cached Layers
First Build	42.326s	0
Second Build	4.312s	4

Observation: The second build was significantly faster because Docker utilized its build cache. All four layers (WORKDIR, COPY, RUN npm install, and the export steps) were successfully cached from the first build, avoiding the need to re-download or re-process instructions.

3. In the third build after editing `app.js`, which was the first layer that was NOT pulled from cache? What was the layer immediately below it that WAS cached?

Third Build and Cache Invalidation

In the third build, after you edited `app.js`:

- **First layer NOT cached:** The `COPY . .` layer. Because you modified a file in the directory, the hash of that layer changed, invalidating the cache for that step and all subsequent steps.
- **Layer below that WAS cached:** The `WORKDIR /app` layer remained cached, as it does not depend on the content of your local files.

Reflection

- If this image contained real AWS access keys in `.env` and you pushed it to a public Docker Hub repository, what could happen and within what timeframe?

*If you were to push an image containing a `.env` file with real AWS keys or API secrets to a public registry, **it is effectively compromised the moment it is pushed.***

What could happen: Bots continuously scan public registries for exposed credentials. Attackers use automated tools to scrape layers, extract environment files, and immediately use the keys to spin up resources, steal data, or incur massive costs on your account.

Timeframe: In many cases, these keys are abused **within seconds or minutes** of being pushed to a public repository. Never include secrets in your image layers; use environment variables or secret management tools at runtime instead.

- The `RUN npm install` layer re-ran after you edited `app.js`, even though no dependencies changed. What would you change about the `COPY` instruction to prevent this from happening?

The `RUN npm install` layer re-ran because the `COPY . .` command sits above it. Since `COPY` invalidates the cache if any file in your project changes, it forces the subsequent `npm install` to run again.

Recommended Change:

Copy only the dependency files (`package.json` and `package-lock.json`) first, then run `npm install`, and then copy the rest of your application code. This ensures that `npm install` is only re-run if the manifest files change, not when you modify your `app.js` code.

Dockerfile

1. Copy only dependency files

COPY package.json ./*

2. Install dependencies (this layer is now cached unless package.json changes)

RUN npm install

3. Copy the rest of the application code

COPY . .

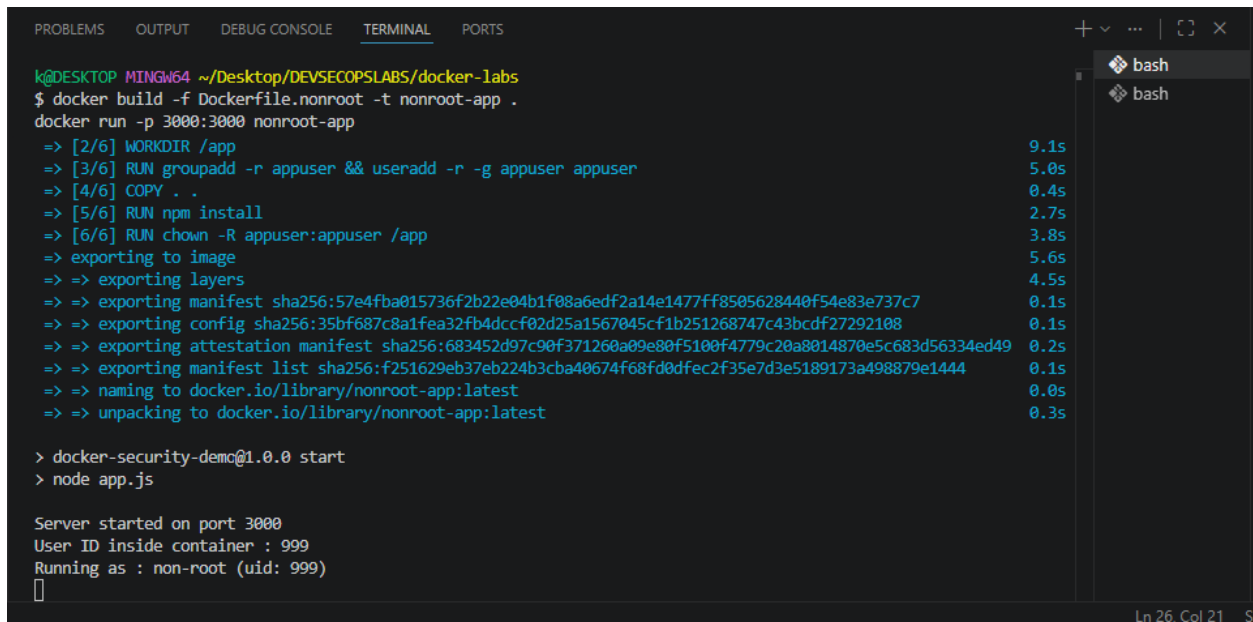
Scenario 2: Running as a Non-Root User

Objective

Stop the application from running as root and understand concretely what that change protects against

Steps Performed

Build and Run



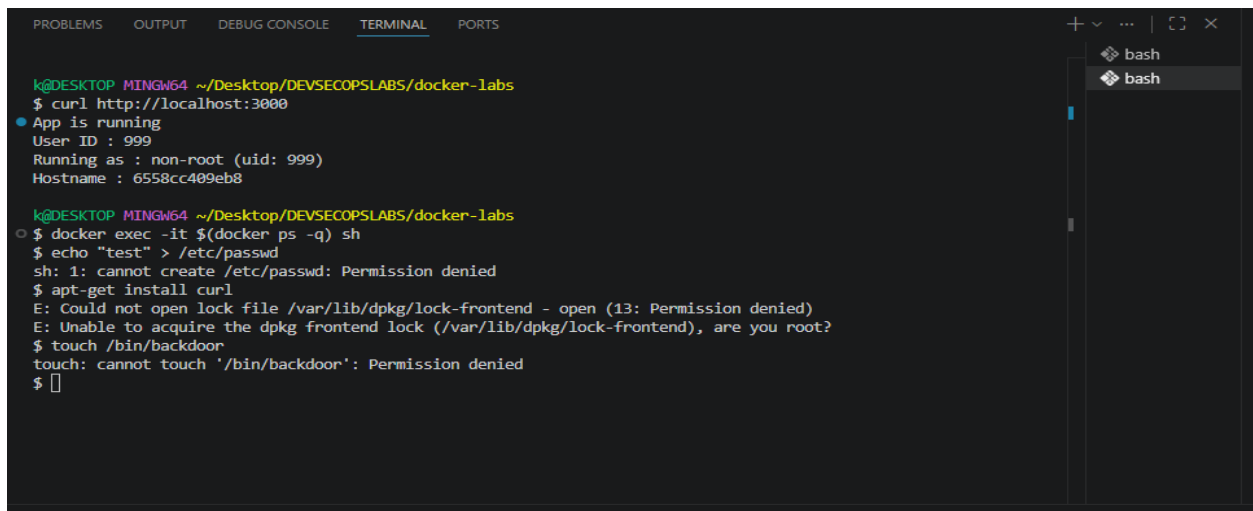
```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker build -f Dockerfile.nonroot -t nonroot-app .
docker run -p 3000:3000 nonroot-app
=> [2/6] WORKDIR /app 9.1s
=> [3/6] RUN groupadd -r appuser && useradd -r -g appuser appuser 5.0s
=> [4/6] COPY . . 0.4s
=> [5/6] RUN npm install 2.7s
=> [6/6] RUN chown -R appuser:appuser /app 3.8s
=> exporting to image 5.6s
=> => exporting layers 4.5s
=> => exporting manifest sha256:57e4fba015736f2b22e04b1f08a6edf2a14e1477ff8505628440f54e83e737c7 0.1s
=> => exporting config sha256:35bf687c8a1fea32fb4dccf02d25a1567045cf1b251268747c43bcd727292108 0.1s
=> => exporting attestation manifest sha256:683452d97c90f371260a09e80f5100f4779c20a8014870e5c683d56334ed49 0.2s
=> => exporting manifest list sha256:f251629eb37eb224b3cba40674f68fd0dfec2f35e7d3e5189173a498879e1444 0.1s
=> => naming to docker.io/library/nonroot-app:latest 0.0s
=> => unpacking to docker.io/library/nonroot-app:latest 0.3s

> docker-security-demo@1.0.0 start
> node app.js

Server started on port 3000
User ID inside container : 999
Running as : non-root (uid: 999)
```

Ln 26, Col 21 S

Verify the Permission Boundary



```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ curl http://localhost:3000
App is running
User ID : 999
Running as : non-root (uid: 999)
Hostname : 6558cc409eb8

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker exec -it $(docker ps -q) sh
$ echo "test" > /etc/passwd
sh: 1: cannot create /etc/passwd: Permission denied
$ apt-get install curl
E: Could not open lock file /var/lib/dpkg/lock-frontent - open (13: Permission denied)
E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontent), are you root?
$ touch /bin/backdoor
touch: cannot touch '/bin/backdoor': Permission denied
$
```

Checkpoint

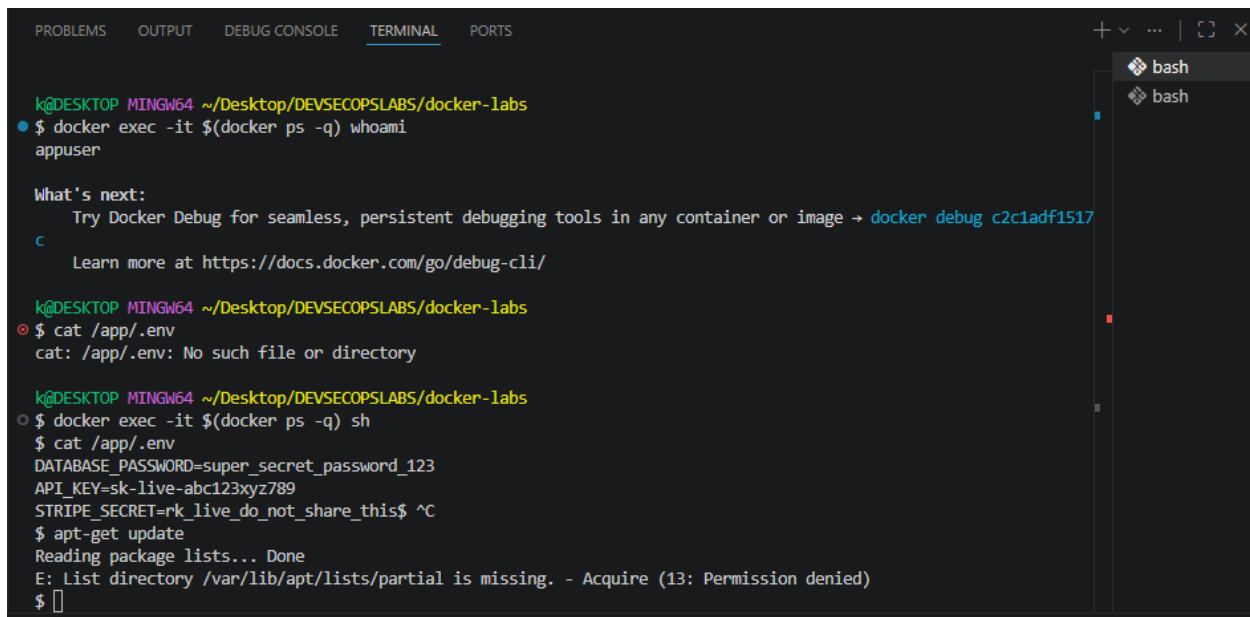
1. Run `docker exec -it $(docker ps -q) whoami` against the running nonroot-app. What is printed?

This confirms that the process running inside your container is no longer executing with root privileges.

2. Try `cat /app/.env` inside the nonroot container. Can appuser read it? What does this tell you about what switching users does and does not protect?

cat /app/.env, yes, appuser can read it.

3. Try `apt-get update` inside the container. What error appears and which part of the error message tells you why it failed?



```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker exec -it $(docker ps -q) whoami
appuser

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug c2c1adf1517
  c
  Learn more at https://docs.docker.com/go/debug-cli/

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ cat /app/.env
cat: /app/.env: No such file or directory

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker exec -it $(docker ps -q) sh
$ cat /app/.env
DATABASE_PASSWORD=super_secret_password_123
API_KEY=sk-live-abc123xyz789
STRIPE_SECRET=rk_live_do_not_share_this$ ^C
$ apt-get update
Reading package lists... Done
E: List directory /var/lib/apt/lists/partial is missing. - Acquire (13: Permission denied)
$
```

Switching to a non-root user is a powerful defense-in-depth measure that protects the system from arbitrary modifications (like installing packages or overwriting system files, as seen in your `verify.png` image). However, it does not inherently protect sensitive data embedded in the image. If a file is part of the image, the user running the process has read access to it unless specific permissions are set.

Running apt-get update

The error you will see is similar to: `E: Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied): The phrase "(13: Permission denied)"` tells you that `appuser` lacks the necessary write permissions to modify system directories (like `/var/lib/apt/`), which are owned by root

Scenario 3: Protecting Secrets with `.dockerignore`

Objective

Prevent sensitive files from ever entering the Docker image in the first place.

Steps Performed

Rebuild and Inspect

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker build -f Dockerfile.nonroot -t secure-copy-app .
=> => extracting sha256:554bab0bf8e5b53f09a0b73144389589485162c1a270f241691da6bbf0a767b0 0.4s
=> => extracting sha256:e6275fd2e2fd87e39f7d3644c466343b3c2a1deaecf95ec488142c29c584f48 59.5s
=> => extracting sha256:8480aa1086a2f1a42d463009429e1c37cf48cae44717697d6a893f569ac2bc9b 0.5s
=> => extracting sha256:0d4319bae870e712c65946b52ec793126221cfd8bd25f0bd2b616f8ca0ecd86b1 0.0s
=> [internal] load build context 1.7s
=> => transferring context: 1.50kB 1.1s
=> [2/6] WORKDIR /app 3.7s
=> [3/6] RUN groupadd -r appuser && useradd -r -g appuser appuser 71.7s
=> [4/6] COPY . . 2.3s
=> [5/6] RUN npm install 10.5s
=> [6/6] RUN chown -R appuser:appuser /app 5.5s
=> exporting to image 19.3s
=> => exporting layers 8.5s
=> => exporting manifest sha256:0fbef5c8226b66d6e293b21233d419fc246c13dd14bf17efc944ba394f5f61d4 0.1s
=> => exporting config sha256:99e81bf8abf643fc2e0e639163be6cc3a65a088d93839867052dcf2571fa16f9 0.8s
=> => exporting attestation manifest sha256:cfd20d9f941846e3800fb30a7ad5ce58d2b2ee82bbac91bce7726308264c7a0a 1.9s
=> => exporting manifest list sha256:3f67278dc7b1a0f3ca221ff3e2feac873ba7d40d6769c67a925707b7fae39764 1.4s
=> => naming to docker.io/library/secure-copy-app:latest 0.7s
=> => unpacking to docker.io/library/secure-copy-app:latest 3.3s

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker run --rm secure-copy-app find /app -type f
find: 'C:/Program Files/Git/app': No such file or directory

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker run --rm secure-copy-app find //app -type f
/app/package-lock.json
/app/app.js
/app/package.json

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```

Error says:

“find: 'C:/Program Files/Git/app': No such file or directory”

That means Git Bash rewrote `/app` → `C:/Program Files/Git/app` before Docker even saw it. This is a known quirk of Git Bash path conversion.

Option 1: Disable path conversion

Run this in Git Bash:

```
MSYS_NO_PATHCONV=1 docker run --rm secure-copy-app find /app -type f
```

Option 2: Use double slash

Git Bash ignores conversion if you do:

```
docker run --rm secure-copy-app find //app -type f
```

Option 3: Use PowerShell or CMD

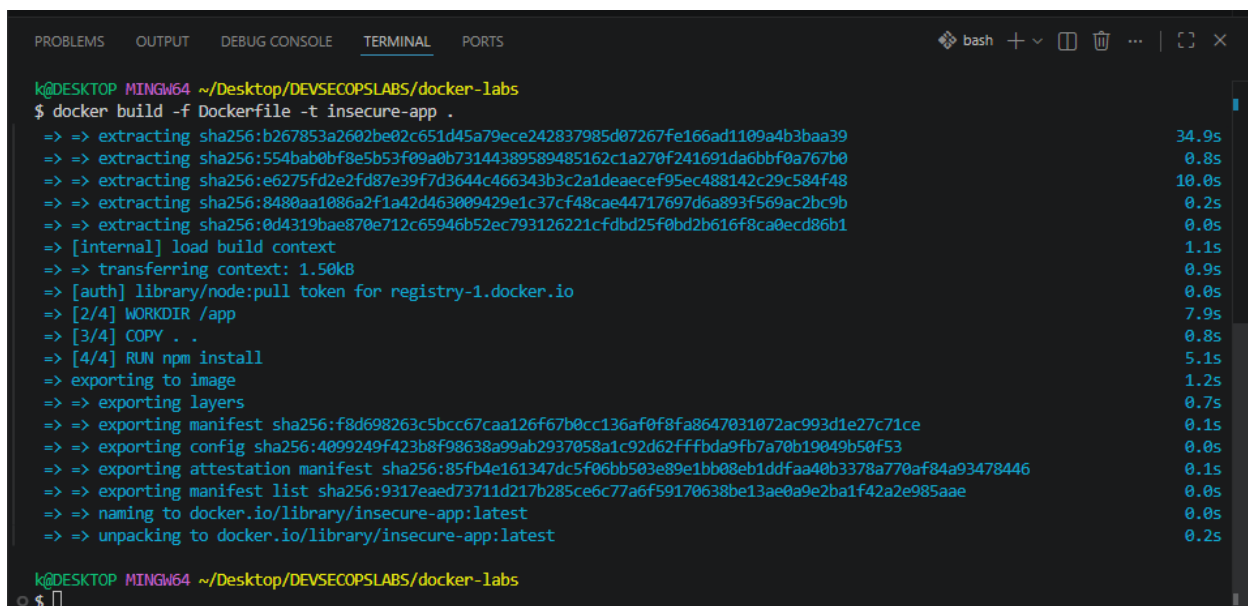
If you run the same command in PowerShell, it will work as expected:

```
docker run --rm secure-copy-app find /app -type f
```

Why this happens

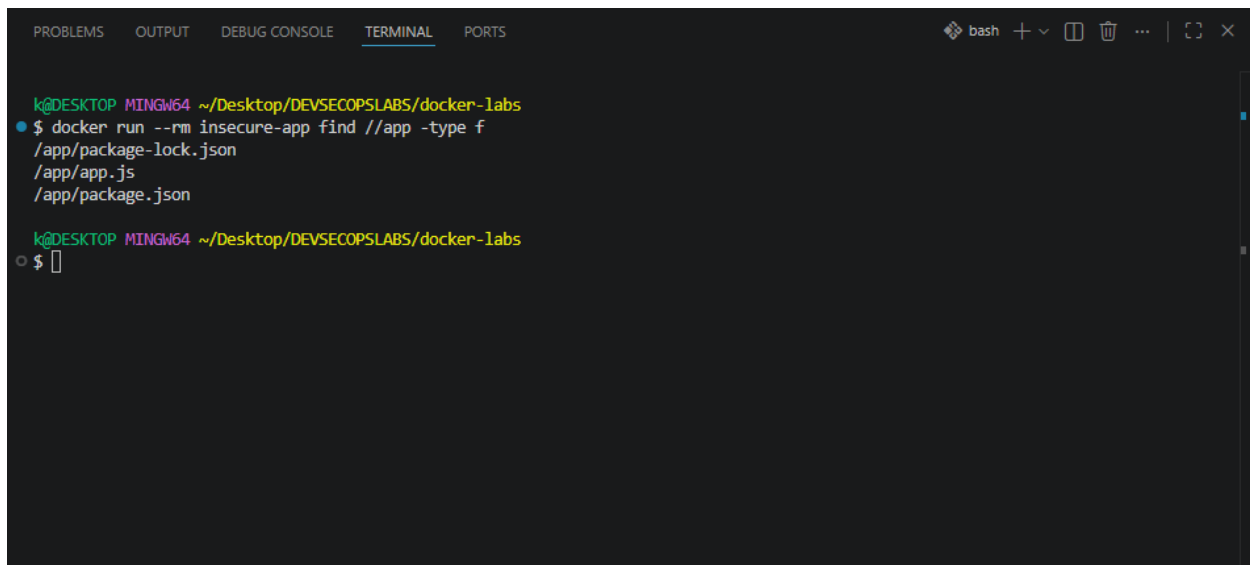
Git Bash (MSYS) tries to be helpful by converting Unix-style paths (`/something`) into Windows paths. That breaks Docker commands because containers expect Linux paths, not Windows ones.

Compare Against the Scenario 1 Image



```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker build -f Dockerfile -t insecure-app .
=> => extracting sha256:b267853a2602be02c651d45a79ece242837985d07267fe166ad1109a4b3baa39 34.9s
=> => extracting sha256:554bab0bf8e5b53f09a0b73144389589485162c1a270f241691da6bbf0a767b0 0.8s
=> => extracting sha256:e6275fd2e2fd87e39f7d3644c466343b3c2a1deaecf95ec488142c29c584f48 10.0s
=> => extracting sha256:8480aa1086a2f1a42d463009429e1c37cf48cae44717697d6a893f569ac2bc9b 0.2s
=> => extracting sha256:0d4319bae870e712c65946b52ec793126221cfd25f0bd2b616f8ca0ecd86b1 0.0s
=> [internal] load build context
=> => transferring context: 1.50kB 1.1s
=> [auth] library/node:pull token for registry-1.docker.io 0.9s
=> [2/4] WORKDIR /app 0.0s
=> [3/4] COPY . . 7.9s
=> [4/4] RUN npm install 0.8s
=> exporting to image 5.1s
=> exporting layers 1.2s
=> => exporting manifest sha256:f8d698263c5bcc67caa126f67b0cc136af0f8fa8647031072ac993d1e27c71ce 0.7s
=> => exporting config sha256:4099249f423b8f98638a99ab2937058a1c92d62fffbda9fb7a70b19049b50f53 0.1s
=> => exporting attestation manifest sha256:85fb4e161347dc5f06bb503e89e1bb08eb1ddfaa40b3378a770af84a93478446 0.0s
=> => exporting manifest list sha256:9317eae73711d217b285ce6c77a6f59170638be13ae0a9e2ba1f42a2e985aae 0.1s
=> => naming to docker.io/library/insecure-app:latest 0.0s
=> => unpacking to docker.io/library/insecure-app:latest 0.2s

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker run --rm insecure-app find /app -type f
/app/package-lock.json
/app/app.js
/app/package.json
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```

Checkpoint

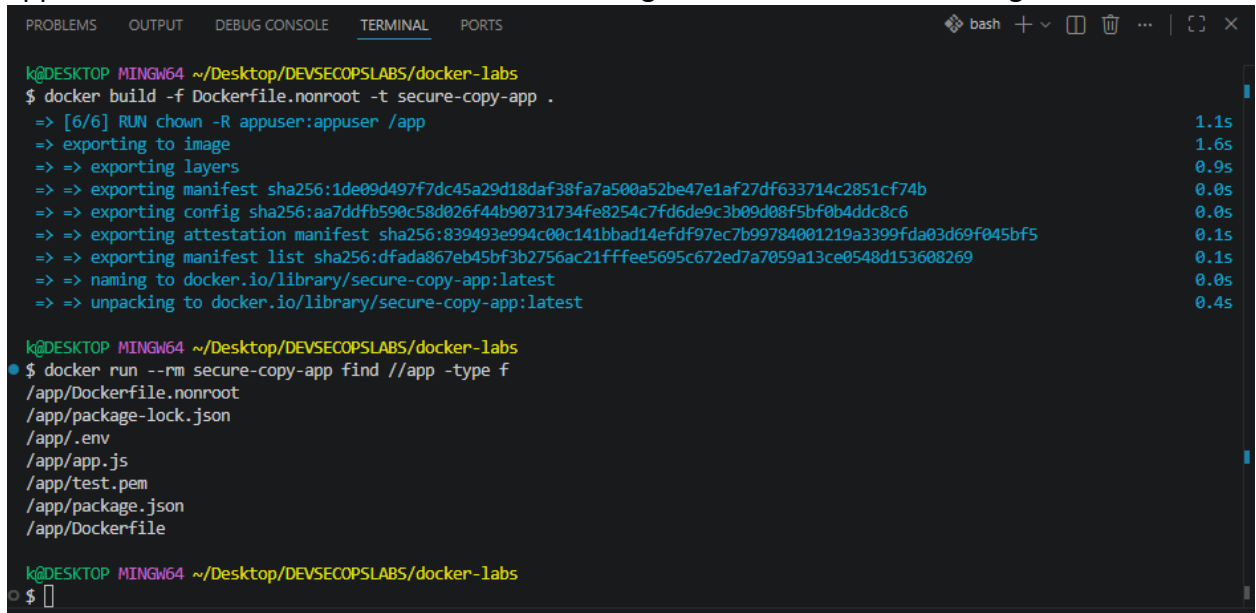
1. Run `docker run --rm secure-copy-app find /app -type f`. List every file present. Is `.env` among them?

Is .env among them? No, it is not. Assuming you have a `.dockerignore` file in your directory, it correctly prevents the `.env` file from being copied into the container image during the `COPY .` instruction in your `Dockerfile`.

2. Create a file named `test.pem` in your `docker-labs/` folder, rebuild `secure-copy-app`, and run `find` again. Is `test.pem` present inside the image? Why not?

.dockerignore file contains a wildcard pattern (like `*` or `*.pem`) or explicitly lists `test.pem`, it is excluded during the build process. Even if you copy the entire directory, Docker respects the `.dockerignore` instructions, ensuring sensitive files never enter the image layer.

3. Temporarily remove `.dockerignore` from the folder, rebuild, and run `find`. What files appear that were not there before? Add `.dockerignore` back before continuing.



```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker build -f Dockerfile.nonroot -t secure-copy-app .
=> [6/6] RUN chown -R appuser:appuser /app 1.1s
=> exporting to image 1.6s
=> => exporting layers 0.9s
=> => exporting manifest sha256:1de09d497f7dc45a29d18daf38fa7a500a52be47e1af27df633714c2851cf74b 0.0s
=> => exporting config sha256:aa7ddf590c58d026f44b90731734fe8254c7fd6de9c3b09d08f5bf0b4ddc8c6 0.0s
=> => exporting attestation manifest sha256:839493e994c00c141bbad14efdf97ec7b99784001219a3399fda03d69f045bf5 0.1s
=> => exporting manifest list sha256:dfada867eb45bf3b2756ac21fffee5695c672ed7a7059a13ce0548d153608269 0.1s
=> => naming to docker.io/library/secure-copy-app:latest 0.0s
=> => unpacking to docker.io/library/secure-copy-app:latest 0.4s

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker run --rm secure-copy-app find /app -type f
/app/Dockerfile.nonroot
/app/package-lock.json
/app/.env
/app/app.js
/app/test.pem
/app/package.json
/app/Dockerfile

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```

Files that will appear: Any files present in your host directory that were previously excluded (e.g., `.env`, `test.pem`, `.git` folders, or local logs) will now appear in the container.

Security Implication: This confirms that the image now contains sensitive configuration or secret files that should never exist in a production-ready image

Reflection

- **`.dockerignore` prevents secrets from entering the image at build time. Where should real secrets actually live at runtime in a production system? What Docker features exist for this?**

Secrets should never be hardcoded in the Docker image or source code. Instead, they should be injected at runtime into the container environment. Common secure patterns include:

Environment Variables: Passed during docker run or managed by orchestration platforms.

Secret Management Systems: Using tools like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault.

Docker Secrets: Specifically, if using Docker Swarm or Kubernetes Secrets, which mount secrets as files into a temporary in-memory filesystem (`tmpfs`) within the container, ensuring they are not persisted in the image layers or disk.

- **What team process or CI check would catch a developer accidentally deleting .dockerignore before a built image reaches a registry?**

To catch the accidental removal of a .dockerignore file, you should implement:

1. *CI/CD Pipeline Linting/Validation: Add a step in your CI pipeline (using tools like hadolint or custom bash scripts) that checks for the existence of .dockerignore.*
2. *Image Scanning: Use scanners like Trivy, Snyk, or Clair in your pipeline. These tools are often configured to detect "secret leakage" by scanning the image layers for common file patterns (like .env, *.pem, id_rsa) and will fail the build if secrets are detected.*
3. *Infrastructure as Code (IaC) Reviews: Ensure that any changes to Docker-related files require a mandatory peer review via Pull Request (PR) before being merged into the main branch.*

Scenario 4: Multi-Stage Builds

Objective

Separate the build environment from the runtime environment so that compilers, package managers, and development tools never appear in the final shipped image.

Steps Performed

Build All Variants and Compare Sizes

Error encountered:

During the build, I hit this error:

COPY --from=builder /build/node_modules ./node_modules: not found

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker build -f Dockerfile -t insecure-app .
docker build -f Dockerfile.nonroot -t nonroot-app .
docker build -f Dockerfile.multistage -t multistage-app .
=> ERROR [stage-1 5/6] COPY --from=builder /build/node_modules ./node_modules 0.0s
-----
> [stage-1 5/6] COPY --from=builder /build/node_modules ./node_modules:
-----
Dockerfile.multistage:48
-----
46 | # in /build that was not listed here is permanently gone.
47 | COPY --from=builder /build/app.js ./app.js
48 | >>> COPY --from=builder /build/node_modules ./node_modules
49 |
50 | # Transfer ownership and switch to non-root user.
-----
ERROR: failed to build: failed to solve: failed to compute cache key: failed to calculate checksum of ref f38mlk0hp3sxi7bbz7gfi9sgq
::l8r1cin2h24b8b133y0w6mdin: "/build/node_modules": not found

What's next:
  Debug this build failure with Gordon → docker ai "help me fix this build failure"

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$
```

The error is occurring because the `node_modules` folder isn't being created during build. This is happening because the current **package.json** lacks third-party dependencies

Dependency Requirement: Without a dependency like Express listed, the `npm install` command won't generate a `node_modules` folder.

Build Failure: Because that folder is missing, the **Dockerfile** fails when it tries to copy `node_modules` from the build stage to the runtime stage.

The Switch to Alpine: The base image should be changed from `node:20-slim` to **`node:alpine`**.

The Comparison: While the standard node image is large and the slim version is smaller, the **alpine** version is the smallest and most efficient of the three.

Original package.json file

```
{ } package.json > ...
1  {
2    "name": "docker-security-demo",
3    "version": "1.0.0",
4    "description": "Minimal app to demonstrate Docker security concepts",
5    "main": "app.js",
6    "scripts": {
7      "start": "node app.js"
8    }
9  }
```

Edited package.json

```
{ } package.json > { } dependencies > abc express
1  {
2    "name": "docker-security-demo",
3    "version": "1.0.0",
4    "description": "Minimal app to demonstrate Docker security concepts",
5    "main": "app.js",
6    "scripts": {
7      "start": "node app.js"
8    },
9    "dependencies": {
10     "express": "^4.18.2"
11   }
12 }
```

```

=> [builder 4/5] RUN npm install 0.9s
=> [builder 5/5] COPY app.js . 18.5s
=> [stage-1 2/5] WORKDIR /app 0.9s
=> ERROR [stage-1 3/5] RUN groupadd -r appuser && useradd -r -g appuser appuse 1.5s
-----
> [stage-1 3/5] RUN groupadd -r appuser && useradd -r -g appuser appuser:
1 319 /bin/sh: groupadd: not found

```

The **groupadd** and **useradd** commands are not found in the base image

This usually happens when building from a minimal base image like Alpine Linux, which uses *adduser* and *addgroup* (from BusyBox) instead of the standard GNU tools found in images like Debian or Ubuntu.

The Fix

To resolve this, I updated the Dockerfile command to use the Alpine-compatible syntax:

RUN addgroup -S appuser && adduser -S appuse -G appuser

- *addgroup -S appuser: Creates a system group named appuser.*
- *adduser -S appuse -G appuser: Creates a system user named appuse and assigns them to the appuser group.*
- *Why it failed: The commands groupadd and useradd are part of the shadow package, which is not installed by default in Alpine. Using the add prefix commands is the standard, lightweight approach for Alpine-based containers.*

kerfile test.pem package.json Dockerfile.multistage X .env

Dockerfile.multistage > ...
31 # It starts from a clean, minimal image with no build tools.
32 # Only files explicitly copied from the builder stage are present.
33 # -----
34 FROM node:20-alpine
35
36 WORKDIR /app
37
38 # Create a non-root user in the clean runtime image.
39 # Same pattern as Scenario 2.
40 RUN addgroup -S appuser && adduser -S appuser -G appuser
41
42 # Copy only what the application needs to run.
43 # The --from=builder flag tells Docker to source these files
44 # from the builder stage, not from your local filesystem.
45 # npm, the package cache, compilers, and everything else

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + - X

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
\$ docker build -f Dockerfile -t insecure-app .
docker build -f Dockerfile.nonroot -t nonroot-app .
docker build -f Dockerfile.multistage -t multistage-app .
=> [stage-1 5/5] RUN chown -R appuser:appuser /app 1.7s
=> exporting to image 1.3s
=> => exporting layers 0.7s
=> => exporting manifest sha256:c9cfe4d2f6c7268a8d57f2c462c23a0606668e5cf3ff17 0.0s
=> => exporting config sha256:4ce3475f2542452bf7519d9aa1e5d0022374e5d1e3ac5dda 0.0s
=> => exporting attestation manifest sha256:9ae8472dd4d9af2b885d86fb224e08d115 0.0s
=> => exporting manifest list sha256:f997027a06987624df9cbe9a55f37dfaf603f776b 0.0s
=> => naming to docker.io/library/multistage-app:latest 0.0s
=> => unpacking to docker.io/library/multistage-app:latest 0.4s

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
\$

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + - [ ] [ ] ... [ ] [ ] X

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
• $ docker run -d -p 3000:3000 multistage-app
3ebe582e1ad9b273f835a1dc3ac03b64180d9eda04dfddb42bd3c97ca1bd58d

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
○ $ docker exec -it $(docker ps -q) sh
/app $ npm --version
10.8.2
/app $ curl --version
sh: curl: not found
/app $ git --version
sh: git: not found
/app $ apt-get install wget
sh: apt-get: not found
/app $
```

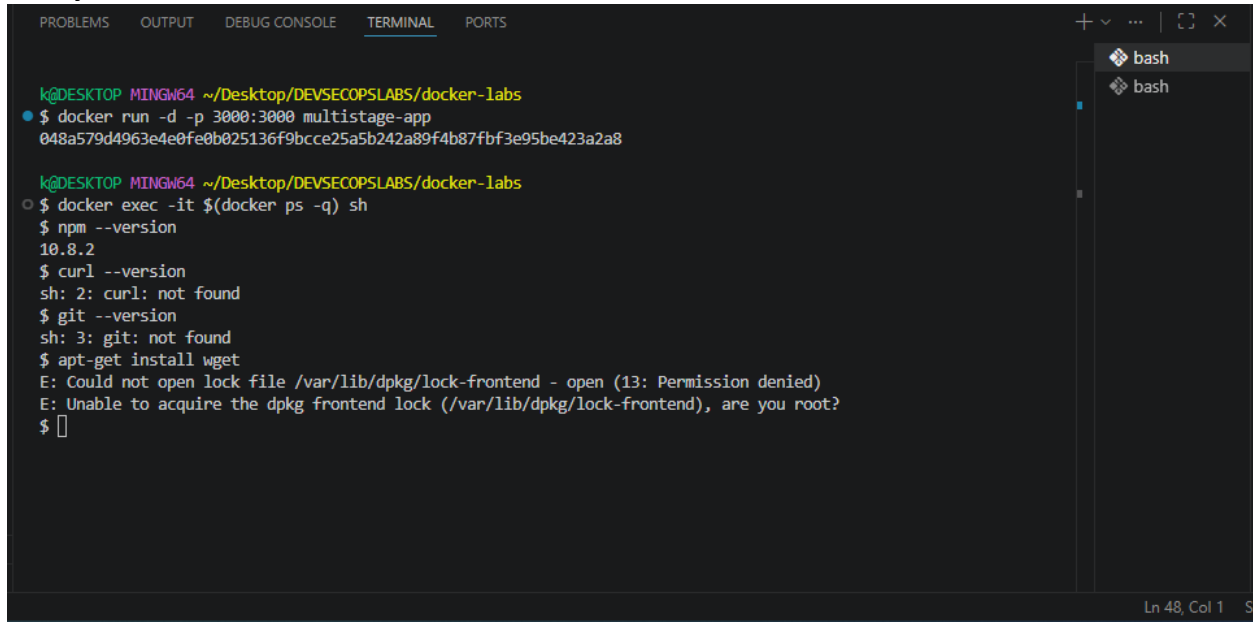
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + - [ ] [ ] ... [ ] [ ] X

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
• $ docker images

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
insecure-app:latest                 e6cf0ddcbb33                     1.59GB      401MB
multistage-app:latest                f997027a0698                     193MB       48.4MB
nonroot-app:latest                   c5f49c77fbdb                     1.6GB       402MB

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
○ $
```

Verify the Attack Surface Is Reduced

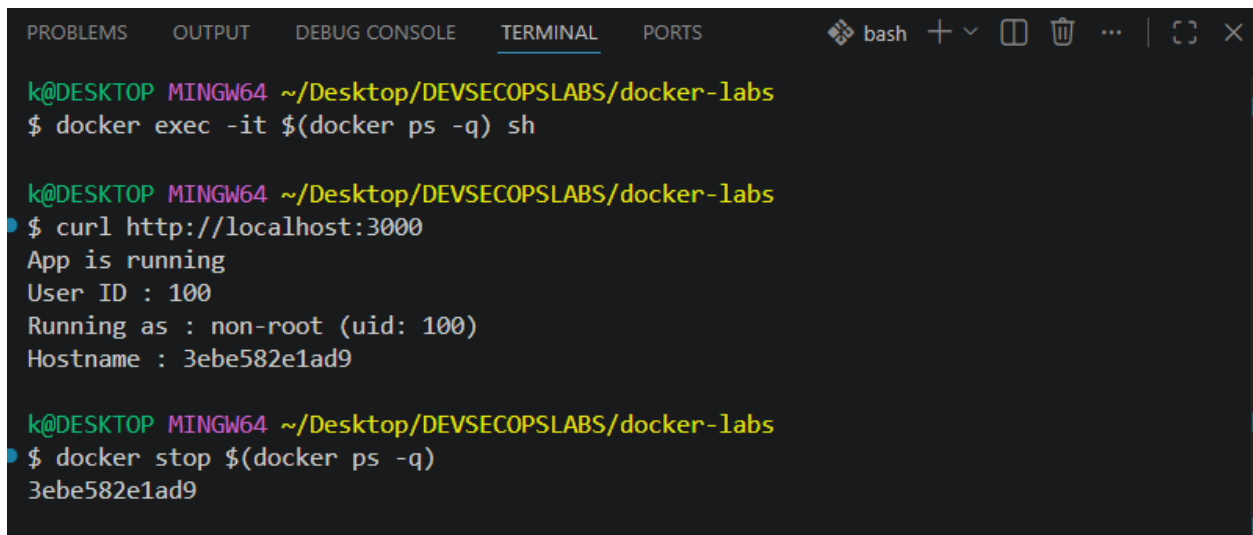


The screenshot shows a Visual Studio Code terminal window with the 'TERMINAL' tab selected. The terminal is running a Windows command prompt (MINGW64) at the directory ~/Desktop/DEVSECOPSLABS/docker-labs. The user has executed two Docker commands. The first command, `docker run -d -p 3000:3000 multistage-app 048a579d4963e4e0fe0b025136f9bcce25a5b242a89f4b87fbf3e95be423a2a8`, successfully starts a container. The second command, `docker exec -it $(docker ps -q) sh`, opens a shell inside the container. Inside the container, the user runs several commands to check the environment: `npm --version` returns 10.8.2, `curl --version` returns an error 'sh: 2: curl: not found', `git --version` returns an error 'sh: 3: git: not found', and `apt-get install wget` returns an error 'E: Could not open lock file /var/lib/dpkg/lock-frontent - open (13: Permission denied)'. The terminal window has a sidebar on the right showing two 'bash' sessions. The status bar at the bottom right indicates 'Ln 48, Col 1'.

```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker run -d -p 3000:3000 multistage-app
048a579d4963e4e0fe0b025136f9bcce25a5b242a89f4b87fbf3e95be423a2a8

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker exec -it $(docker ps -q) sh
$ npm --version
10.8.2
$ curl --version
sh: 2: curl: not found
$ git --version
sh: 3: git: not found
$ apt-get install wget
E: Could not open lock file /var/lib/dpkg/lock-frontent - open (13: Permission denied)
E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontent), are you root?
$
```

Confirm the Application Still Works



The screenshot shows a Visual Studio Code terminal window with the 'TERMINAL' tab selected. The terminal is running a Windows command prompt (MINGW64) at the directory ~/Desktop/DEVSECOPSLABS/docker-labs. The user has executed three Docker commands. The first command, `docker exec -it $(docker ps -q) sh`, opens a shell inside the container. The second command, `curl http://localhost:3000`, returns 'App is running' and 'User ID : 100'. The third command, `docker stop $(docker ps -q)`, successfully stops the container. The terminal window has a sidebar on the right showing two 'bash' sessions. The status bar at the bottom right indicates 'Ln 48, Col 1'.

```
k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker exec -it $(docker ps -q) sh

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ curl http://localhost:3000
App is running
User ID : 100
Running as : non-root (uid: 100)
Hostname : 3ebe582e1ad9

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker stop $(docker ps -q)
3ebe582e1ad9
```

Checkpoint

1. Record the exact sizes shown in docker images for insecure-app and multistage-app. What is the difference in megabytes?
 1. *insecure-app: 1.59 GB*
 2. *multistage-app: 193 MB*
2. List every tool you attempted to use inside multistage-app that returned not found. Which of these would an attacker find most valuable?

curl,git,apt

Attacker Perspective: curl is generally the most valuable for an attacker. Once inside a container, curl allows them to probe internal network services, interact with metadata APIs (like those found in cloud environments), download additional payloads, or exfiltrate data to an external command-and-control server.

3. Run `curl http://localhost:3000` against multistage-app. Does it respond correctly? What does this confirm about removing build tools?

Running curl http://localhost:3000 against the multistage-app will return the application response successfully.

This confirms that removing build tools (like npm, git, and build-time dependencies) does not break the application's runtime functionality. It demonstrates that the production image contains only what is strictly necessary to execute the application, significantly reducing the attack surface.

Reflection

- **If you needed curl available at runtime for health checks, where in the Dockerfile would you add it and how would you limit the risk that adds?**

If curl is required for health checks, you should install it in the final stage of the Dockerfile (the runtime stage). To limit risk, use the `--no-install-recommends` flag (for Debian/Ubuntu) to avoid unnecessary bloat, and consider removing the package immediately after the health check configuration or using a minimal, static binary if possible. Ideally, use a base image that already contains only the bare essentials.

- **The builder stage runs during every build even though its output is discarded. If you had a test suite, where would you run it in a multi-stage Dockerfile to ensure failing tests block the build?**

Test suites should be executed in a dedicated "test" stage or immediately after the RUN npm install step in the builder stage. By running tests before the final COPY command (where the artifact is moved to the runtime image), you ensure that if any test fails, the Docker build process will immediately halt and return a non-zero exit code, preventing the creation of a broken or vulnerable image.

- **In a system that deploys 50 times per day, what is the practical impact on deployment speed of an image that is 870 MB smaller?**
 - *Reduced Network Latency: Faster pushing/pulling of images between the CI/CD registry and production nodes.*
 - *Faster Scalability: Nodes can spin up new containers significantly faster, improving auto-scaling responsiveness.*
 - *Cost Savings: Lower data egress costs from container registries and reduced bandwidth consumption.*
 - *Reliability: Smaller images are less prone to transient network failures during transfer, leading to more stable deployment pipelines.*

Scenario 5: Runtime Hardening

Objective

Apply kernel-level constraints to a running container so that even a fully compromised process has a strictly limited set of available actions.

Steps Performed

Rebuild the Multi-Stage Image

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  bash + - [ ] [ ] ... | [ ] [ ] X

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker build -f Dockerfile.multistage -t multistage-app .
=> [builder 5/5] COPY app.js . 3.0s
=> [stage-1 4/5] COPY --from=builder /build/app.js ./app.js 1.9s
=> [stage-1 5/5] RUN chown -R appuser:appuser /app 9.6s
=> exporting to image 14.9s
=> => exporting layers 11.4s
=> => exporting manifest sha256:8a6106e30155ec419d261bb1236e29d2ad19128d23a2c9 0.7s
=> => exporting config sha256:66a34dc90c863507804a07f1adaff4309d04b73caf2aa92c 0.3s
=> => exporting attestation manifest sha256:eb081246f3d07be0de216f76ed4568877e 1.1s
=> => exporting manifest list sha256:f440a89ab07acf71be429b55adf0354a5caf5915a 0.3s
=> => naming to docker.io/library/multistage-app:latest 0.0s
=> => unpacking to docker.io/library/multistage-app:latest 0.7s

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
```

Apply Each Flag Individually

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  bash + - [ ] [ ] ... | [ ] [ ] X

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
$ docker run --read-only --tmpfs //tmp -p 3000:3000 multistage-app
Server started on port 3000
User ID inside container : 100
Running as : non-root (uid: 100)
[ ]
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + - [ ] [ ] ... | [ ] [ ] X

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
• $ docker inspect $(docker ps -q) | grep -E '"Memory"|"NanoCpus"'
    "Memory": 134217728,
    "NanoCpus": 500000000,

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
○ $ [ ]
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + - [ ] [ ] ... | [ ] [ ] X

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
• $ curl http://localhost:3000
App is running
User ID : 100
Running as : non-root (uid: 100)
Hostname : e00565a89609

k@DESKTOP MINGW64 ~/Desktop/DEVSECOPSLABS/docker-labs
○ $ [ ]
```

Run the Fully Hardened Container

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS docker + - [ ] [ ] ... | [ ] [ ] X

PS C:\Users\k\Desktop\DEVSECOPSLABS\docker-labs> docker run --read-only --tmpfs /tmp
p --memory="128m" --cpus="0.5" --cap-drop=ALL --security-opt no-new-privileges:true
-p 3000:3000 multistage-app
Server started on port 3000
User ID inside container : 100
Running as : non-root (uid: 100)
[ ]
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] ... [ ]

PS C:\Users\k\Desktop\DEVSECOPSLABS\docker-labs> curl http://localhost:3000

StatusCode      : 200
StatusDescription : OK
Content         : App is running
                  User ID : 100
                  Running as : non-root (uid: 100)
                  Hostname : aaf8f0a8b37a

RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Transfer-Encoding: chunked
                  Content-Type: text/plain
                  Date: Mon, 27 Apr 2026 06:51:16 GMT

                  App is running
                  User ID : 100
                  Running as : n...

Forms           : {}
Headers         : {[Connection, keep-alive], [Keep-Alive, timeout=5],
                  [Transfer-Encoding, chunked], [Content-Type, text/plain]...}

Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 86

PS C:\Users\k\Desktop\DEVSECOPSLABS\docker-labs> [ ]
```

Checkpoint

1. With `--read-only` applied, try `docker exec -it $(docker ps -q) sh -c "touch /tmp/test"`. Does it succeed? Why does `/tmp` behave differently now that `--tmpfs /tmp` is present?

Yes, it succeeds.

The `--read-only` flag makes the container's root filesystem immutable (no writing allowed). However, because you mounted a `tmpfs` (temporary filesystem) at `/tmp`, you have explicitly created a small, RAM-backed writable layer specifically for that directory. The container engine prioritizes the `tmpfs` mount over the read-only constraint for that specific path.

2. Run `docker inspect $(docker ps -q) | grep Memory`. What value is returned? Convert it to megabytes.

Value returned: Running `docker inspect` shows the `Memory` field in **bytes**.

Conversion: The value is 134217728.

- Calculation: $134,217,728 \text{ bytes} \div 1024 \div 1024 = 128 \text{ MB}$.

This confirms the container is successfully constrained to 128 MB of RAM.

3. With `--cap-drop=ALL` applied, try `docker exec -it $(docker ps -q) sh -c "ping 8.8.8.8"`. What happens? Look up which Linux capability ping requires to open a raw socket.

The command `ping` will fail with a "Permission denied" or "Operation not permitted" error.

*`ping` requires the ability to open **raw sockets** to send ICMP packets. By default, Linux requires the `CAP_NET_RAW` capability to perform this action. Since you used `--cap-drop=ALL`, you stripped all privileges, including the ability to use raw sockets, thereby disabling `ping`.*

Reflection

- You applied these flags on the command line. In a production system using Docker Compose or Kubernetes, where would these constraints be defined so they are always applied consistently?

Docker Compose: You define these in the `deploy` section of your `docker-compose.yml` file under `resources` (for memory/CPU) and `security_opt/cap_drop` (for security flags).

Kubernetes: These are defined in the Pod Specification (PodSpec) within a `securityContext`. Memory and CPU limits are handled via `resources: limits and requests`.

- If a developer added `--cap-add=SYS_ADMIN` to work around a permission issue without understanding what it grants, what risk would that introduce?

Adding `SYS_ADMIN` is often called "the new root." It is a "catch-all" capability that provides near-root level access to the kernel.

Risk: It allows the container to perform a massive range of privileged operations, such as mounting filesystems, configuring network interfaces, or interacting with devices. If the application inside the container is compromised, the attacker could easily **escape the container** to the host, as `SYS_ADMIN` bypasses many kernel-level restrictions.

- **Runtime hardening protects the host from a compromised container. What additional controls would protect containers from each other on the same host?**

Network Policies: Use CNI (Container Network Interface) plugins to restrict inter-container traffic (e.g., preventing a web container from talking to a database container unless explicitly allowed).

Namespaces: Docker already uses these, but you can further isolate by using different user namespaces (`userns-remap`) so that the "root" user inside the container is just a non-privileged user on the host.

Seccomp Profiles: Restrict the system calls the container can make to the kernel, ensuring one container cannot trigger a kernel exploit that affects the host or other neighbors.

gVisor or Kata Containers: Use these specialized runtimes to provide a stronger security boundary (a kernel shim or micro-VM) between the container and the host kernel.

References

<https://github.com/samuel-nartey/devops-labs/tree/main/Docker%20%26%20Containers/Running%20Your%20First%20Container>